```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("/content/train.csv")
```

```
df.head()
```

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775	pixel776	pixel777	pix
0	1	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	
1	0	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	
2	1	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	
3	4	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	
4	0	0	0	0	0	0	0	0	0	0	...	0.0	0.0	0.0	0.0	

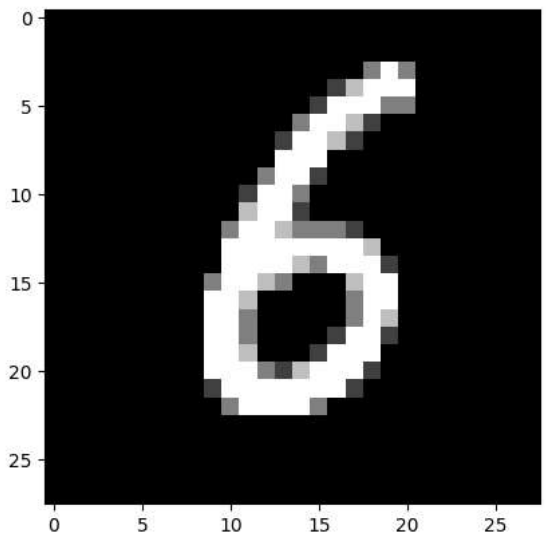
5 rows × 785 columns

```
df.shape
```

```
(17784, 785)
```

```
# image show row 570 and col from 1 becz 1st onwards is label
plt.imshow(df.iloc[570, 1:].values.reshape(28,28), cmap='gray')
```

```
<matplotlib.image.AxesImage at 0x7fb0da8b3f50>
```




```
X = df.iloc[:, 1:] # image
y = df.iloc[:, 0] #label
```

```
from sklearn.model_selection import train_test_split
# training (80%) and testing (20%)
# random_state=42 → ensures same split every time
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state = 42)
```

```
X_train.shape
```

```
(14227, 784)
```

```
# K-Nearest Neighbors (KNN) classification algorithm from the scikit-learn library.
# It is used to classify data points based on the majority class of their nearest neighbors.
# It helps by accurately classifying new data points based on patterns learned from nearby similar data.
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier()
```

```
print(X_train.shape) # 2D (samples, features)
print(y_train.shape) # 1D (labels)
```

```
(14227, 784)
(14227,)
```

```
print(X_train.isnull().sum().sum())
```

```
604
```

```
X_train = X_train.fillna(0)
```

```
# trains the KNN model using the training data (X_train) and their corresponding labels (y_train).
knn.fit(X_train, y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
import time
start = time.time()
y_pred = knn.predict(X_test)
print(time.time() - start)
```

```
5.0872883796691895
```

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

```
0.9589541748664605
```